15 · DDD、事件驱动、CQRS 与 GraphQL 设计

项	内容
文档编号	DT-PdM-15
依据	DDD（限界上下文/聚合/领域事件）；CQRS（Command Query Responsibility Segregation）；CloudEvents 1.0；GraphQL 2021 规范
版本	v1.0（新增）

---

1. 为什么 v1 必须换地基

v1 的文章是"好的"，但它是单厂、集中、同步、单一读侧的模型。放到雅迪场景立刻撞墙：

真实约束	v1 的做法	撞墙结果
20 基地弱网	同步 REST 调用中心	请求超时、数据丢失、体验不可用
高频写 + 重读	一套库	看板查询拖垮采集写入
前端一次要 6 个域数据	6 次 REST 往返	首屏 10 秒以上
诊断证据要不可抵赖	普通数据库表	证据可被改，责任说不清
领域逻辑复杂（会诊/经济性）	按技术层 Controller/Service/Dao	每加一个基地规则都要改核心

换地基，不是打补丁。

---

2. 限界上下文划分（8 个）

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■                       Diting-PdM                                ■
■                                                                 ■
■  ■■■■■■■■■■■■■■■■   ■■■■■■■■■■■■■■■■   ■■■■■■■■■■■■■■■■     ■
■  ■ ① ■■■■■  ■   ■ ② ■■■■■  ■   ■ ③ ■■■■■  ■    ■
■  ■ Asset       ■■■■■■ Monitoring  ■■■■■■ Diagnosis   ■    ■
■  ■ ■■/■■/■■■   ■ ■■/■■/■■■   ■ HI/■■/■■  ■    ■
■  ■■■■■■■■■■■■■■■■   ■■■■■■■■■■■■■■■■   ■■■■■■■■■■■■■■■■     ■
■        ■                  ■                  ■            ■
■        ▼                  ▼                  ▼            ■
■  ■■■■■■■■■■■■■■■■   ■■■■■■■■■■■■■■■■   ■■■■■■■■■■■■■■■■     ■
■  ■ ⑥ ■■■■■  ■■■■■■ ④ ■■■■■  ■   ■ ⑤ ■■■■■  ■    ■
■  ■ Knowledge   ■    ■ Prognosis    ■■■■■■ Decision    ■    ■
■  ■ ■■/■■/■■■   ■ RUL/■■/OOD ■   ■ ■■/■■■   ■    ■
■  ■■■■■■■■■■■■■■■■   ■■■■■■■■■■■■■■■■   ■■■■■■■■■■■■■■■■     ■
■                                              ■           ■
■        ■■■■■■■■■■■■■■■■   ■■■■■■■■■■■■■■■■   ■            ■
■        ■ ⑦ ■■■■■■ ■■■■■■ ⑧ ■■■■■  ■■■■■             ■
■        ■ Agent Runtime■   ■ Ops/■■/■■ ■                ■
■        ■■■■■■■■■■■■■■■■   ■■■■■■■■■■■■■■■■                ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

2.1 上下文职责与 Owner

#	限界上下文	核心聚合	语言	团队
①	Asset 资产	Asset、Component、MeasurementPoint	Java（组织树/权限复杂）	平台
②	Monitoring 监测	AcquisitionJob、Series、QualityScore	Rust（高频写/DSP）	平台

#	限界上下文	核心聚合	语言	团队
③	Diagnosis 诊断	Diagnosis、HealthIndex、FaultHypothesis	Python（算法）	算法
④	Prognosis 预后	RulEstimate、DegradationCurve	Python	算法
⑤	Decision 决策	Consultation、RecommendationCard	Python（会诊/经济性）	算法
⑥	Knowledge 知识	FaultMode、Judgement、Skill、Case	Java + PG/Nebula	算法
⑦	Agent 智能体	AgentSession、ToolCall、EvidenceChain	Python（Harness）	算法
⑧	Ops 运营	Alert、Incident、WorkOrderDraft	Go（高并发事件）	平台

2.2 上下文映射（Context Mapping）

关系	模式	说明
①→②	客户-供应商（Customer-Supplier）	资产上下文物模型是监测的上游
②→③	防腐层（ACL）	监测的"原始特征"经 ACL 转为诊断的"证据"
③→④	共享内核	共享 `Diagnosis` 值对象
③+④→⑤	客户-供应商	决策消费诊断与预后
⑥→③④⑤	发布语言（Published Language）	判据库是全集团唯一判据，集体订阅
⑦→③④⑤⑥⑧	开放主机服务（OHS）	通过 MCP 工具暴露，不直接调内部
⑤→⑧	ACL	决策输出经 ACL 转工单（隔离 EAM 字段）
⑧→外部 EAM	ACL	防腐层隔离外部模型

> 别让 EAM 的字段名污染你的领域模型。这是 ACL 存在的唯一理由，也是多集团迁移时最救命的一条。

---

3. 聚合设计（重点：三个核心聚合）

3.1 Diagnosis（诊断）

```
■■■■Diagnosis
  ■ diagnosis_id■UUID■
  ■ asset_id, component_id
  ■ window■■■■■
  ■ health_index■■■■■hi, level, contributions[]■
  ■ fault_hypotheses[]■■■■■mode, confidence, evidence_refs[]■
  ■ degraded■■■ + ■■■
  ■ evidence_chain■Evidence ■■■■■■■■■
  ■ model_ref■■■■■ + ■■■■■■
  ■ status■DRAFT | FINALIZED | SUPERSEDED■

■■■■Invariant■■
  I1  evidence_chain ■■■■ FINALIZED
  I2  FINALIZED ■■■■■■■■ SUPERSEDED■■■■■■■
  I3  fault_hypotheses ■ mode ■■■■■ Knowledge ■■■■■■■■
  I4  degraded=true ■ confidence ■■■■ 0.6
```

为什么 Evidence 是实体且不可变：因为"证据可回放"（P2 原则）要求证据一旦生成就不能改。改一次，整个审计链失效。

3.2 RecommendationCard（方案卡）

```
■■■■RecommendationCard
  ■ card_id, diagnosis_id
  ■ options[]■■■■window, downtime_h, crew, spares[], cost{}, risk■
  ■ economics■avoided_loss, total_cost, net_benefit■
  ■ conflicts[]
  ■ confirmations[]■■■, ■, ■■, ■■■
  ■ status■PENDING | CONFIRMED | REJECTED | EXPIRED | CONVERTED■

■■■■
  I1  options ■■ 1 ■
  I2  ■■ must ■ spare_ready ■ cost
  I3  net_benefit = avoided_loss − Σcost
  I4  PENDING → CONVERTED ■■■■■■■■ confirmations
  I5  REJECTED ■■■ reason_code
```

3.3 EvidenceChain（证据链，事件溯源的载体）

```
■■ Evidence■
  ■ ev_id, type, ref■■■■■■, window, value
  ■ source■■■■■/■■■■/■■■■■■■
  ■ ts, actor
  ■ hash■■■■■ Evidence ■■■■

E_n = SHA256(E_{n-1} || canonical_json(Ev_n))
```

这是本项目最"贵"的设计，也是最值钱的。有了它，"AI 说错了谁负责"不再是道德问题，而是可举证的技术问题。

---

4. 领域事件目录（CloudEvents）

4.1 事件命名规范

```
com.yadea.pdm.<context>.<entity>.<pastTenseVerb>.v<N>
```

4.2 事件清单

事件	上下文	触发	订阅方
`...asset.registered.v1`	Asset	新设备注册	Knowledge, Ops
`...asset.decommissioned.v1`	Asset	设备退役	Knowledge
`...monitoring.qualityDropped.v1`	Monitoring	质量分 < 60	Diagnosis, Ops
`...monitoring.channelStale.v1`	Monitoring	测点断流	Ops
`...diagnosis.finalized.v1`	Diagnosis	诊断定稿	Prognosis, Decision, Agent
`...diagnosis.superseded.v1`	Diagnosis	新诊断顶替旧	Agent, Ops
`...prognosis.rulUpdated.v1`	Prognosis	RUL 变化 > 20%	Decision
`...prognosis.oodDetected.v1`	Prognosis	工况越界	Ops

事件	上下文	触发	订阅方
`...decision.consultationStarted.v1`	Decision	会诊开始	Agent
`...decision.roleOpinionSubmitted.v1`	Decision	角色意见	Decision（内部）
`...decision.conflictDetected.v1`	Decision	一致性 < 0.6	Ops（人工）
`...decision.cardCreated.v1`	Decision	方案卡生成	Ops, 审批流
`...decision.cardConfirmed.v1`	Decision	人工确认	Ops
`...decision.cardRejected.v1`	Decision	拒绝（含原因码）	Decision, Knowledge
`...ops.alertRaised.v1`	Ops	告警生成	通知, Agent
`...ops.alertEscalated.v1`	Ops	超时升级	通知
`...ops.workOrderCreated.v1`	Ops	工单草稿创建	EAM, 审计
`...ops.workOrderConfirmed.v1`	Ops	EAM 回执	Decision
`...knowledge.criteriaPublished.v1`	Knowledge	判据发布	Diagnosis（全基地）
`...knowledge.modelApproved.v1`	Knowledge	模型准入	Diagnosis

4.3 事件信封（CloudEvents + 多租户扩展）

```
{
  "specversion": "1.0",
  "id": "01J9Z8X2K3",
  "type": "com.yadea.pdm.decision.cardCreated.v1",
  "source": "/pdm/decision-service",
  "subject": "site-a/KA-101",
  "time": "2026-10-21T06:31:12Z",
  "tenantid": "site-a",
  "groupid": "SZ01",
  "traceid": "01J9Z...",
  "causationid": "01J9Z...",
  "datacontenttype": "application/json",
  "data": { }

}
```

> `tenantid` 必须出现在信封一级，不能只放进 data。Pulsar 的 namespace 虽然已经隔离了 topic，但订阅端做二次过滤时需要它，且对未来跨主题审计有用。

---

5. 关键流程 Saga：会诊 → 申请 → 执行

```
[■■] diagnosis.finalized.v1 (level ≥ orange)
   ■
   ▼
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

```
■ Saga: MaintenanceAdvisorySaga                                          ■
■                                                                         ■
■ S1  Decision ■■■■                                                     ■
■       → consultationStarted                                               ■
■       → ■■ 4 ■■■timeout 30s■                                          ■
■                                                                         ■
■ S2  Evaluator ■■                                                      ■
■       ■ ■■ → S3                                                       ■
■       ■ ■■■ → conflictDetected → [■■■■] → S3 ■ ■■                   ■
■                                                                         ■
■ S3  ■■■■■                                                            ■
■       → cardCreated                                                      ■
■       → ■■ PENDING■■■ 48h                                              ■
■                                                                         ■
■ S4  ■■■■                                                              ■
■       ■ cardConfirmed■■■ token, 2h ■■■                                ■
■       ■ ■■ → EXPIRED → ■■■■■ → ■■                                     ■
■                                                                         ■
■ S5  ■■■■■■■EAM■■■■■■                                            ■
■       ■ ■■ → workOrderCreated■■■■■→ S7                                ■
■       ■ ■■ → ■■ ×3■■■■■■→ ■■■ → ■■■■ → S6                      ■
■                                                                         ■
■ S6  ■■■■■"■■■"■■■■■■■■■                                       ■
■                                                                         ■
■ S7  ■■ + ■■■■ → workOrderConfirmed                                    ■
■       → Saga ■■■■■■■                                                 ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

■■■■■
· S5 ■■■■■ S3■■■■■■■■■■■■■■■
· ■■■■■■■■■■■■■■■■
· Saga ■■■■■■■■■■■■ + ■■ span■
```

为什么用 Saga 而不是分布式事务：会诊链条横跨 5 个上下文和 1 个外部系统，两阶段提交在这里既不可行也不必要——唯一必须强一致的是"方案卡状态与工单号"这一对，用回执校验（Stop Hook）保证就够了。

6. CQRS：读写分离

6.1 两侧边界

```
■■■■Write■                              ■■■■Read■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■         ■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■■/■■/■■/■■              ■■■■■■■/■■/API
■■■■                                   ■■■■
 ingest → feature → diagnosis                ■■■■ → ■■ → ■■■
 → prognosis → decision                       ■■■■■■■■ < 3s■
■■■                                     ■■■
 · TDengine■■■■■■■■                  · PG■■■■■■■■■■■■■
 · MinIO/NFS■■■■■■                    · TiDB + TiFlash■■■■■■■
 · PG■■■■■■■■                         · Redis■■■■■
 · Pulsar■■■■■                            · NebulaGraph■■■■■■
 · NebulaGraph■■■■■■■■                  · Milvus■■■■■■
■■■■■■■■■■■■■■            ■■■■■■■■■■■■■
```

6.2 读模型投影器（Projector）

```
■■ → Projector → ■■■■■■■■ sequence ■■■

■■■■■■
1. ■■■■■■asset.registered → read_mm_assets
2. ■■■■■diagnosis.finalized × N → read_mm_asset_health■■■■■■ HI■
3. ■■■■■■■rulUpdated × N → read_mm_trend■TiDB■■■/■■■■
4. ■■■■■■× 20 ■■ → read_mm_group_benchmark■TiFlash ■■■

■■■■■
· ■■■■■ sequence■Pulsar messageId■
· ■■■■■ processed_events(event_id) ■■■■
· ■■■■■■■■■■■■■■
```

6.3 延迟预算（最终一致的可接受边界）

读模型	允许延迟	超期后果
设备当前 HI	≤ 5 s	报告可能不新
告警列表	≤ 3 s	用户体验下降
跨基地对标	≤ 60 s	不敏感
趋势图	≤ 5 min	不敏感
审计链	实时（同步写）	不可延迟

> 审计链路同步写，不走投影。这是唯一例外——因为它不是"读模型"，它是"证据"。

---

7. GraphQL BFF 层

7.1 为什么需要

前端看板一次需要：设备信息 + 当前 HI + 最近告警 + RUL + 方案卡 + 趋势。v1 的做法是 6 次 REST 调用 + 前端拼装，首屏慢且弱网下极易失败。

7.2 Schema 设计（节选）

```
type Query {
  # ■■/■■■■
  assetHealthList(
    tenantId: ID!
    filter: AssetFilter
    page: Page          # ■■■■■■ BFF ■■■■■■■■■■■
  ): AssetHealthConnection!

  # ■■■■■■■■■■■■■ N+1■
  assetDetail(assetId: ID!): AssetDetail

  # ■■■■■■■ TiFlash ■■■■■■
  groupBenchmark(model: String!, metric: Metric!, from: DateTime!): [BenchmarkPoint!]!
}

type AssetDetail {
  id: ID!
  profile: AssetProfile
  health: HealthIndex
  rul: RulEstimate
  alerts(last: Int = 10): [Alert!]!
  recommendations(status: [CardStatus!]): [RecommendationCard!]!
  trend(from: DateTime!, to: DateTime!): TrendSeries
}

# ■■■■■■■■■■■■■■■■■
type AssetHealth @authz(require: "asset:read", scope: TENANT)
type RecommendationCard @authz(require: "card:read", scope: TENANT)
```

7.3 三条硬规则

规则	原因
禁止客户端拼任意深度查询	查询成本攻击；BFF 需深度限制（≤ 10）+ 复杂度评分
禁止 N+1	必须用 DataLoader 批量化，否则一次查询打爆下游
禁止把数据库字段直接暴露	GraphQL schema 是"发布语言"，不能随数据库 schema 漂移

7.4 与 REST / MCP 的分工

入口	用途	消费者
GraphQL	前端看板、复杂聚合查询	Web / App / 大屏

入口	用途	消费者
REST	简单资源操作、运维接口、外部系统	运维脚本 / EAM / 集成
MCP	智能体工具调用	Agent Harness

三者并存，各有明确消费者，不要把某个入口强行统一成唯一入口。

---

8. 事件总线选型：复用 Pulsar（已实测存在）

8.1 实测结论

```
10.167.2.175/176 ■■ infra ■■■■■■■■
  pulsar            10.110.176.161:6650■broker■
  pulsar-broker     6650/8080
  pulsar-bookie-0/1 3181■ledgers + journal ■ bookie■
  pulsar-zookeeper  2181
```

8.2 为什么选它而不是 Kafka

维度	Pulsar（存量）	Kafka（需新建）
多租户	原生 tenant/namespace/topic 三级，权限可精细授权	仅 topic 前缀约定，靠 ACL
分层存储	bookkeeper 自动分层	需自建 tiered storage
弱网友好	有 geo-replication、可延迟确认	弱
运维现状	已在跑，零部署成本	新增一套 broker + ZK
团队熟悉度	已有运维经验	需要学习

> Pulsar 的 tenant/namespace 模型对"20 个基地"是天然匹配：每个基地一个 tenant，报警/诊断/工单各占 namespace，供应商一个独立 tenant 且授权最小。这是选它的决定性理由，不是因为它更流行。

8.3 接入规范

```
Pulsar URL: pulsar://pulsar-broker.infra:6650
Admin URL: http://pulsar-broker.infra:8080

■■■■■
  tenant/pdm                            # PdM ■
    namespace/pdm.site-a                # ■■■■■
    namespace/pdm.site-b
    namespace/pdm.group                 # ■■■■■■■■■■■■■■
    namespace/pdm.supplier              # ■■■■■■■■■■
    namespace/pdm.audit                 # ■■■■■■■■■■■■

Topic■
  persistent://pdm/site-a/diagnosis.finalized
  persistent://pdm/site-a/alert.raised
  persistent://pdm/group/criteria.published

■■■
  ■■■■■■■■■■■■■■■ projector-asset-health■
  ■■■Shared■■■■■■■■ Failover■■■■■■■■■■
```

---

9. 反模式清单（这些错我见过人犯）

反模式	症状	正确做法
把领域事件当日志用	事件里塞调试信息，订阅方被迫解析字符串	事件是契约，schema 版本化，字段只增不改

反模式	症状	正确做法
投影器里做业务判断	读模型开始"算结论"，与命令侧逻辑分叉	投影器只做搬运，判断留给领域服务
GraphQL 直接透传数据库	一个 schema 改动穿三层到 DBA	schema 是发布语言，加投影层
Saga 没有幂等	重试产生 3 张工单	幂等键 + 去重表 + 回执校验
最终一致当借口	数据不一致就说"最终一致"	定义每类数据的延迟预算，超期告警
DDD 全量铺开	8 个上下文写 8 套完整分层，工期翻倍	只在诊断/决策域做厚模型，监测域薄层
聚合做太大	Diagnosis 聚合包含整个设备历史	聚合边界按"一致性需求"划，不按"方便"划
事件风暴	一个动作发 20 个事件，订阅方雪崩	一个业务动作原则上一个事件

---

10. 落地顺序建议

阶段	先做什么	后做什么
Ph1	Pulsar 接入 + 事件信封 + 审计事件流	读模型投影
Ph2	判据发布事件 + diagnosis.finalized	跨基地对标投影
Ph3	会诊 Saga + GraphQL BFF	供应商域

不要一上来就全量 DDD + 全量 CQRS。 先打通"事件能可靠送达 + 幂等消费"，这是地基；地基不牢，上面都是沙。